

COMP 403

Parallel and distributed processing

Lecture 1

21-9-2026

Course Information

- **Course Code:** COMP 403
- **Course Title:** Parallel and Distributed Processing
- **Credit Hours:** 3
 - **Lecture hours:** 2
 - **Lab. hours:** 2
- **Final Exam (duration):** 3 Hours
- **Total Marks:** 150
 - 105 Final Exam.
 - 37 Mid-Term and Lab. Exam.
 - 8 Oral Exam.

Textbook

- Introduction to Parallel Computing, Z. J. Czech, Cambridge University Press, 2017.

Course Objective

- Introduction to the basic concepts.
- Parallel and distributed systems.
- Basic models of parallel computation.
- Elementary parallel algorithms.
- Performance evaluations of parallel algorithms
 - Time complexity, **speedup**, cost, **efficiency**.
- Designing parallel algorithms.

Outline of Lecture 1

- Introduction and Basic Concepts
 - Sequential Program
 - Parallel Program
 - Sequential Process
 - Synchronous and Asynchronous Execution
- Correctness of Concurrent Programs
- Some Problems in Concurrent Programs
 - The Critical Section Problem
 - Semaphores.
 - The Producer and Consumer Problem
 - Using Bounded-buffer to Solve the producer and Consumer Problem

Sequential Program, SP

- SP describes how to solve a computational problem in a **sequential computer**:
 - For example, the traveling salesman problem:
 - **number of cities, distances between each pair of cities**
 - The solution is a **closed route of the minimum length** of the salesman passing through **every** city exactly **once**.
- SP is a **sequence of instructions** that solves the problem by transforming the input data into the output data.
- A sequential program is executed by a **single processor**.

Parallel Program, PP

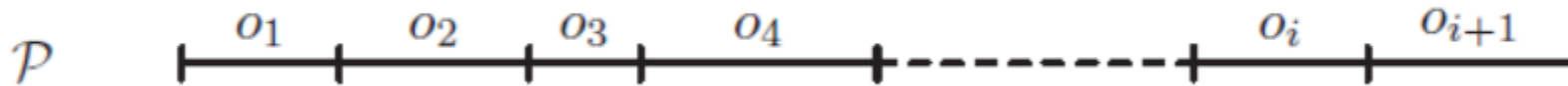
- If there are **more processors** available to solve the problem, the problem can be **partitioned into subproblems** that may be solved in parallel.
- The solution to the original problem is a **composition of solutions to the subproblems**.
- Each component is a traditional sequential program called a **computational task** (or **task**).
- A single task is performed as a sequential (series) process.

Why Parallel Computing?

- **Accelerate computation** which results in solving complex problems in shorter time.
- Solve **large problems**
 - by breaking them into **smaller pieces** that can run in parallel.

Sequential Process

- A **single task** is performed as a **sequential process**- an ordered sequence of **operations**:
 - $o_1, o_2, o_3, \dots$
- A **sequential process** results from execution a **single instruction sequence**.
- **One operation at a time**: the next operation begins **only after** the previous one completes.

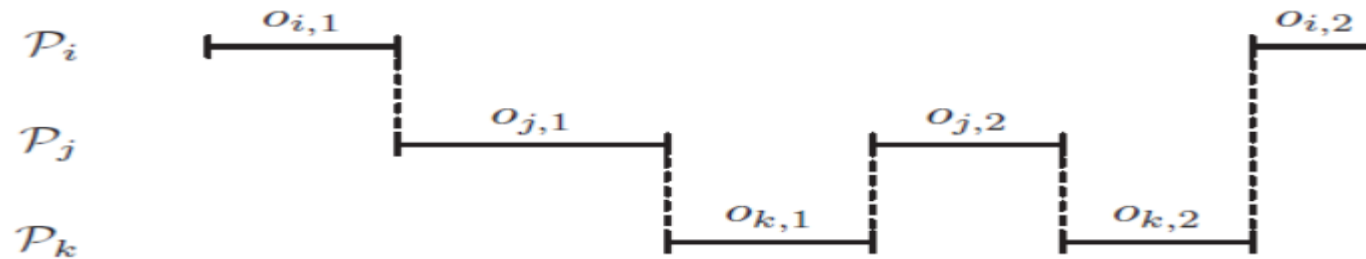


Synchronous Execution

- **Synchronous:** operations are **time-locked** — the caller waits for each operation to finish before proceeding, which **forces** the sequential order of execution.

Concurrent Processes & Asynchronous Execution

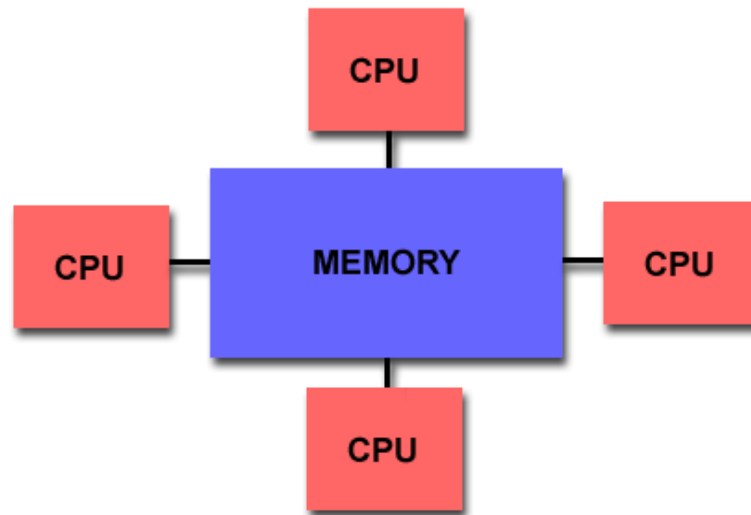
- **Concurrent processes:** sequential processes performed **simultaneously** and **asynchronously**, in which execution can overlap in time:



- Each process is sequential internally, but the processes run **independently of each other**.
- Operations of different processes can occur in **any relative order**.
- **Asynchronous:** operations are **not synchronized in time** — no waiting between processes, so their executions can **interleave or overlap** rather than follow one fixed order.

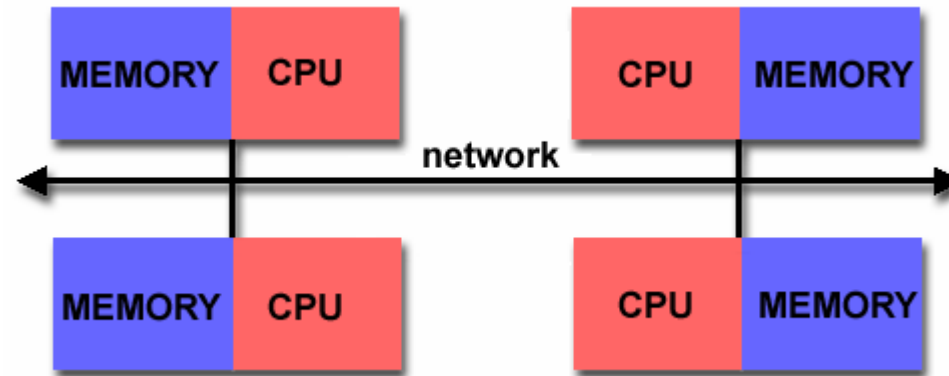
Communication between processes

- **Shared memory**: the ability for all processors to access all memory as global address space.



Communication Between Processes (cont.)

- **Distributed memory** systems require a communication **network** to connect inter-processor memory.
- **Processors** have their own **local memory**. Memory addresses in one processor do not map to another processor, so there is no concept of global address space across all processors.



Distributed Program

- A **Distributed program** is a computer program designed to run across multiple computers (or nodes) connected **via a network**.
- The programs **work together** to perform tasks by **communicating and coordinating through message passing**, rather than relying on a single machine.
- Key focus areas:
 - **Sharing common resources and hardware**
 - **Hardware and software fault tolerance** — if some system components fail, degradation in functionality should be **as small as possible**.

Correctness of Sequential Programs

- Sequential program **is correct** if the desired relations are satisfied between the **input data and output data** that are results of computation.
- The correctness of a sequential program is expressed by a sentence of the form:

$$\{p\} S \{q\}$$

S : program
p : precondition
q : postcondition

- The **precondition** specifies the conditions that are satisfied by the input data (or by the state of memory with which execution of the program begins)
- The **postcondition** specifies the desired conditions to be met by the results of computation (or by the state of memory once the program has been executed)
- Program transforms the computer's memory from a specified initial state to a required final state.

Correctness of Sequential Programs (cont.)

Sequential program S is correct if:

1. S is **partially correct**: if for every terminating execution of S with the input data satisfying precondition p , the output data satisfy postcondition q .
2. S is **totally correct**: if it is partially correct and every execution of S with the input data satisfying precondition p **terminates**.
3. Partial correctness does not consider whether the computation of program S **terminates or not**.

Example

- $\{ x > 0 \} \ x := x * 2; \ \{ x \text{ is even} \}$ correct
- $\{ x = 5 \} \ x := x + 4; \ \{ x = 10 \}$ incorrect
- $\{ k = 5 \} \ k := k + 1 \ \{ k = 6 \}$ correct

Correctness of Concurrent Programs

- The correctness of concurrent programs' tasks (which is sequential programs) must be **demonstrated**.
- Additional properties: **safety** and **liveness** must be proved.

Competing for Shared Resources

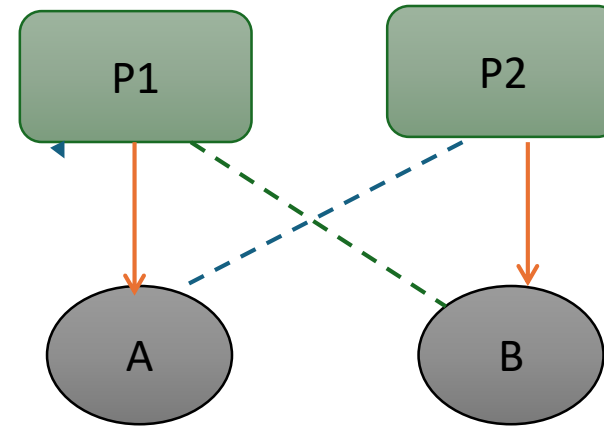
- Concurrent processes often compete for access to **shared resources**, such as:
 - variables in shared memory,
 - files,
 - disk storage,
 - I/O devices
- There are 2 **safety properties**:
 - Mutual exclusion and
 - Free of deadlocks

Safety properties

Property	Definition
Mutual exclusion	At any time, no resource is used by more than one process
Freedom from deadlock	Processes cannot end up permanently blocked, each waiting on the other

The Deadlock Problem

- The deadlock is a situation when two or more processes do not terminate, because they cannot continue their action.
- A classic example:
 - Processes: **P1** and **P2**
 - Resources: **A** and **B**
- Both P_1 and P_2 need both resources, **A** and **B**, to complete their work



- Assume P_1 and P_2 issued their requests and got access to **A** and **B**, respectively.
 - They are now deadlocked when asking for the second resource, because process P_1 holds resource **A**, that is needed by process P_2 , and process P_2 holds resource **B** that is needed by process P_1 .
- Result:** Neither process can proceed — they will **not terminate**.

Liveness Property

Definition:

- During **each concurrent implementation** of a program, a **required condition will be satisfied at some point**.

Example— **competing for a printer**:

- If any process issues a print request, then **at some point it will be assigned the printer**.

Consequence:

- **No individual starvation** — no process is left waiting forever because the resource is never allocated to it.

Problems of Concurrent Programming

The Critical Section Problem

- Definition
 - Occurs when a group of **processes compete** for the **resource**, where at any given time **only one** process can have access to it.
- Resources of this type:
 - **variables in shared memory**
 - **database records**
 - **files**
 - **physical devices (printers, ...)**
- Critical section:
 - The **fragment of a process** (or a task) in which it makes use of a **resource**
- Ensuring exclusive:
 - At any time, **only one** task executes its **critical section**

Critical Section: Example with Two Tasks

Scenario: T_1 and T_2 compete for a resource.

T_1 needs to use the resource:

1. **Acquire permission** — the resource must be assigned to T_1 .
2. **Use the resource** in its **critical section**.
3. **Release the resource** after completing the section.

When T_1 releases the resource

- The resource can now be assigned to T_2 .
- If T_2 requests the resource while T_1 is using it:
 - T_2 must **wait** until T_1 releases it.
- The **critical section** problem can be solved employing **semaphores**.

Semaphores

- **Semaphore** s is a compound data structure with two fields:
 - $s.w$: nonnegative integer values,
 - $s.q$: sets of tasks (processes).
- **Example:**
 - s : semaphore $:= (1, \phi)$
- Two types of semaphores:
 1. **General semaphore:** $s.w$ is any nonnegative integer value
 2. **Binary semaphore:** $s.w$ is only 0 or 1 (**1** : free , **0** : used by others)

```
type semaphore is  
  record  
     $w$ : natural;  
     $q$ : queue;  
  end record;
```

Semaphores Operations

wait(s): If $s.w > 0$, **then** $s.w := s.w - 1$,
else suspend execution of the task performing operation *wait* and add it to set $s.q$.

The task added to set $s.q$ is said to be **blocked on semaphore s** .

signal(s): If set $s.q$ of tasks is nonempty,
then delete and unblock one of the tasks from $s.q$ and resume its execution,
else $s.w := s.w + 1$.

Notes

- The operations *wait* and *signal* are atomic; the actions within these operations cannot be interleaved with any other instructions.
- The set of blocked tasks on a semaphore is organized in a FIFO queue.
- Semaphores must be initialized before use.

Solving Critical Sections using Binary Semaphore

semaphore declaration

type *semaphore* **is**

record

w: *natural*;

q: queue;

end record;

Solving Critical Sections using Binary Semaphore (cont.)

```
1  s: semaphore := (1, ∅); -- binary semaphore
2  task  $T_1$ ; -- task specifications
3  task  $T_2$ ;
4  task body  $T_1$  is
5  begin
6      loop
7           wait(s); -- pre-protocol
8          CRITICAL SECTION
9          signal(s); -- post-protocol
10         -- remainder of the task
11     end loop;
12 end  $T_1$ ;
```

```
13 task body  $T_2$  is
14 begin
15     loop
16         wait(s); -- pre-protocol
17         CRITICAL SECTION
18         signal(s); -- post-protocol
19         -- remainder of the task
20     end loop;
21 end  $T_2$ ;
```



Mutual Exclusion Guaranteed

- The critical section solution has the property of **mutual exclusion**:
- **Suppose** T_1 and T_2 want to execute their critical sections **simultaneously**.
- **Then**:
 1. Both perform **wait(s)**.
 2. Wait(s) is **atomic** (by definition) — so **only one** task enters the critical section.
 3. The **second task is blocked** and placed in the queue.
- If a task attempts to enter while the section is in use:
 - It is **blocked**, because the integer component of s is 0
- **Conclusion**: The solution satisfies **mutual exclusion** — at most one task is ever inside its critical section.

Freedom of Deadlock

Claim: The critical section solution using **binary semaphores** is **free of deadlock**.

Proof by contradiction:

1. Assume **both** T_1 and T_2 are blocked by **wait(s)**.
2. Then **$s.w = 0$** must hold — meaning **one task is executing its critical section**.
3. This **contradicts** the assumption that both are blocked.
4. Therefore, **both tasks cannot be blocked at the same time** — no deadlock.

Critical Sections for n tasks

```
task  $T_i$ ;  
task body  $T_i$  is  
begin  
  loop  
     $wait(s)$ ;  
    -- critical section  
     $signal(s)$ ;  
    -- remainder of the task  
  end loop;  
end  $T_i$ ;
```

The Producer and Consumer Problem

A task, called the **producer**, generates data and passes it to another task, called the **consumer**:

- **Producer** — generates data and passes it to another task.
- **Consumer** — receives and processes that data.

Two solutions for their interaction:

1. Synchronous communication
2. Asynchronous communication

Synchronous Communication

- When a chunk is created, the producer **sends it directly** to the consumer, who **receives and processes it immediately**.
- **Both tasks must be ready** — the producer to send, the consumer to receive.
- **Idle scenarios** — either task may be idle:
 - Producer ready to send, but consumer **not ready to receive** → producer **must wait**.
 - Consumer ready to receive, but producer **has not yet produced** → consumer **must wait**.

Asynchronous Communication

- Uses a **buffer** to store data chunks produced by the producer.
- The consumer retrieves data from the buffer **when ready**.
- **Advantages of buffering:**
 - Tasks operate **independently**, without direct coordination.
 - Instantaneous processing speeds may vary, but **average speeds should align**.

Solving the Producer and Consumer Problem with an unbounded buffer

The declaration:

```
buffer: array(0 .. $\infty$ ) of data_type; -- unbounded buffer  
  in, out: natural := 0; -- buffer indices  
  nonempty: semaphore := (0,  $\emptyset$ );  
  -- general semaphore indicating how many chunks of data are in the buffer
```

Solving the Producer and Consumer Problem with an unbounded buffer (cont.)

```
task Producer;  
task body Producer is  
    data: data_type; -- auxiliary variable containing  
                        -- a chunk of data  
  
    begin  
        loop  
            Produce data;  
            buffer(in) := data; -- insert data into the buffer  
            in := in + 1;  
            signal (nonempty); -- signal data insertion  
        end loop;  
    end Producer;
```

```
task Consumer;  
task body Consumer is  
    data: data_type; -- auxiliary variable  
  
    begin  
        loop  
            wait(nonempty); -- check if there are data in the buffer  
            data := buffer(out); -- retrieve data from the buffer  
            out := out + 1;  
            Consume data;  
        end loop;  
    end Consumer;
```

Blocking the Consumer with semaphore *nonempty*

- If the buffer is empty and the **consumer tries** to read
 - Its operation must be **blocked** until data appears in the buffer.
- To ensure this, a general semaphore *nonempty* is used,
 - Its integer component = **number of data chunks in the buffer**.
- **Producer's role (after inserting a data chunk):**
 - $\text{buffer}(\text{in}) := \text{data}; \text{in} := \text{in} + 1;$
 - $\text{signal}(\text{nonempty});$
 - increases the integer component of *nonempty* by 1
 - resumes the execution of the consumer task if it is waiting in the queue for inserting data.

Consumer: *wait(nonempty)*

- To get data from the buffer, the **consumer** performs *wait(nonempty)*:
 - **Case 1: $\text{nonempty} > 0$**
 - Semaphore is **decremented by 1**, data are **retrieved**.
 $\text{nonempty} := \text{nonempty} - 1;$
 $\text{data} := \text{buffer}(\text{out}); \text{out} := \text{out} + 1;$
 - **Case 2: $\text{nonempty} = 0$**
 - The consumer task is **blocked** until data are inserted into the buffer.

Semaphore *nonempty* vs. Semaphore *s*

- Their roles differ:

	Semaphore <i>s</i>	Semaphore <i>nonempty</i>
Purpose	Synchronizes access to a shared resource	Signals data availability in the buffer
Blocks when	Resource is busy	Buffer is empty
Resumes when	Resource becomes free	Data are inserted

- What **synchronization means**:
 - One task **acquiring knowledge** about the state of execution of another task.
 - **Example**, task T_1 performing performs **wait(s)**
 - if the resource is **free** → assigned to T_1 .
 - Otherwise → T_1 is **blocked** until the resource becomes available.
 - **Consequence**: T_1 can **infer** whether T_2 is currently using the resource.

Condition Synchronization with *nonempty*

- The semaphore *nonempty* indicates whether the producer has inserted data into the buffer.
- Depending on the **buffer state**:
 - $\text{nonempty.w} > 0 \rightarrow$ data is available
 - $\text{nonempty.w} = 0 \rightarrow$ buffer is empty
- This use of semaphore is referred to as **condition** (or **event**) **synchronization**.

Condition synchronization is essential when:

- A task can perform an operation correctly **only if** another task has already executed its operation, **or**
- A **specific event has occurred**.

Problem with unbounded buffer

- **Memory exhaustion**: in practice, unbounded buffers are **theoretical**; if the producer is much faster than the consumer, memory could overflow.